PROJECT REVIEW 3

AuxFlow: Static Analysis for Multi-Pass Document Compilation

1. Title Page

Project title	AuxFlow: Static Analysis for Multi-Pass Document Compilation
Student name	Upayan Mazumder
Register number	24BDS0367
Programme	B.Tech CSE (Data Science)
Department	School of Computer Science and Engineering (SCOPE)
College	Vellore Institute of Technology, Vellore, Tamil Nadu, India
Academic year	2026-2027
Project guide	Banupriya R

2. Abstract

Cross-references, contents pages, and bibliographies can require more than one LaTeX run. Each run reads auxiliary files from the preceding run. Build tools usually repeat the engine until those files appear stable or a run limit is reached. A document with feedback between page references and layout may not settle.

AuxFlow is a static analysis tool and compiler front end for a declared subset of LaTeX. The parser identifies references and generated lists, then the Auxiliary Reference Graph (ARG) records individual facts and their dependencies. Taint analysis checks whether a reference value controls the amount of content. AuxFlow reports CERTIFIED, BENIGN_CYCLIC, AMPLIFYING, or UNKNOWN before compilation. The planner selects engine runs, and the runtime verifier compares the resulting auxiliary files.

The evaluation used 15 documents and 7 baselines, with 10 repetitions per document and baseline (1,050 containerized runs). The predicted bound met or exceeded ground truth for all 7 currently certified documents. Two non-convergent documents were AMPLIFYING and four were UNKNOWN. UNKNOWN does not establish divergence. Analysis took 1.3 ms at the median and 10.4 ms at most. Guards added after the first measurements changed three declared false negatives from CERTIFIED to UNKNOWN.

3. Introduction

Background

LaTeX writes labels to an auxiliary file during one run. A reference can read those labels on the next run. Contents pages and bibliographies add further dependencies. The tools examined here use different stopping tests: latexmk limits itself to 5 runs, Tectonic to 6, ConTeXt to 9, and Typst to 5 iterations. latexrun documents no limit.

Problem statement

The stopping tests examined in this report do not derive a pass bound from the document's source. A fixed schedule can end before references settle. An engine warning can miss changes to a generated list. Reference text can also change line breaks and page numbers, so a document may alter the dependencies that the analysis tries to describe. Static pass numbering assumes a fixed graph. AuxFlow therefore verifies its predicted plan against an engine run.

Objectives

- Classify a document before an engine run and show the source locations of potentially amplifying cycles.

- Calculate a conservative pass bound when the analysis certifies a document.
- Build from that classification and compare auxiliary files after each engine run.
- Compare the results with five deployed stopping tests on a 15-document corpus, including cases that expose limitations.

4. Literature Review

Work	What it contributes	Relation to AuxFlow
Bochmann 1976 and Alblas 1981	They study how to schedule attribute evaluation across passes. Alblas distinguishes simple and pure multi-pass evaluation. Raiha and Ukkonen (1981) show that finding a minimum pass assignment is NP-complete.	AuxFlow numbers individual facts and calculates a conservative bound. It does not claim the fewest possible passes.
Farrow 1986, Jones 1990, Magnusson and Hedin 2007	They study circular attribute grammars. Jones uses strongly connected components to separate cycles. Magnusson and Hedin require finite height and monotone functions for convergence, but their evaluator does not check those conditions.	AuxFlow uses components to report cycles. Its influence analysis tests a restricted form of the finite-height condition.
Hecht and Ullman 1975, Kam and Ullman 1976	They relate iterative data-flow rounds to graph depth.	AuxFlow uses a similar bound: P runs for propagation and one further run in the static schedule.
Mokhov, Mitchell and Peyton Jones 2018, Erdweg et al. 2015 (Pluto)	These works distinguish static and dynamic dependencies in build systems. Pluto handles LaTeX cycles dynamically and questions whether LaTeX dependencies can be found statically in general.	AuxFlow limits the source language. It reports UNKNOWN for unsupported constructs instead of assuming their effects.
latexmk, latexrun, Tectonic, ConTeXt, Typst, Gaspar 2014, Madje 2022	The tools use different stopping tests. Gaspar gives a document that does not settle. Madje discusses why arbitrary program stability is undecidable and proposes a capped check with an error location.	The tools supply stopping-test baselines. Gaspar's loop informs corpus member b08.

The earlier work supplies ways to number passes and analyze cycles, but TeX documents can change their own dependency structure. I used fact instances rather than fact kinds so that the graph records where a particular label, reference, or generated entry comes from. The corpus tests when that model helps and where it misses feedback.

5. Methodology

Proposed system and architecture

Parsing, graph construction, classification, and planning use only source files. The auxflow run command executes the plan and checks the files TeX writes.

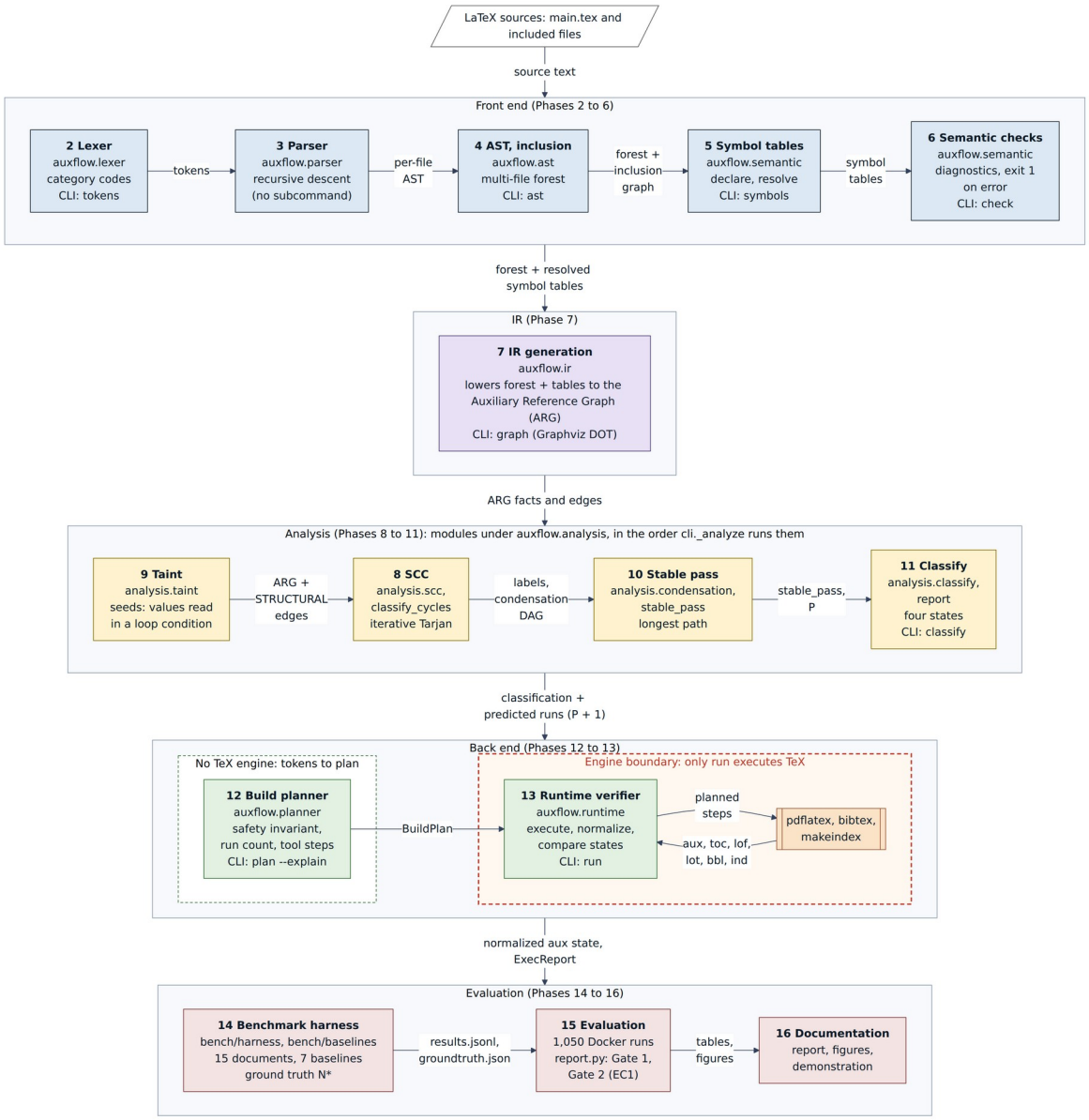


Figure 1. AuxFlow compiler pipeline

Data model

The Auxiliary Reference Graph (ARG) has a node for each label, reference, or other fact instance. Each node records its kind, key, component (exists or value), and source span. Edges record the dependency, pass lag, reason, and type: AUX_ROUNDTRIP, TOOL, LAYOUT, or STRUCTURAL.

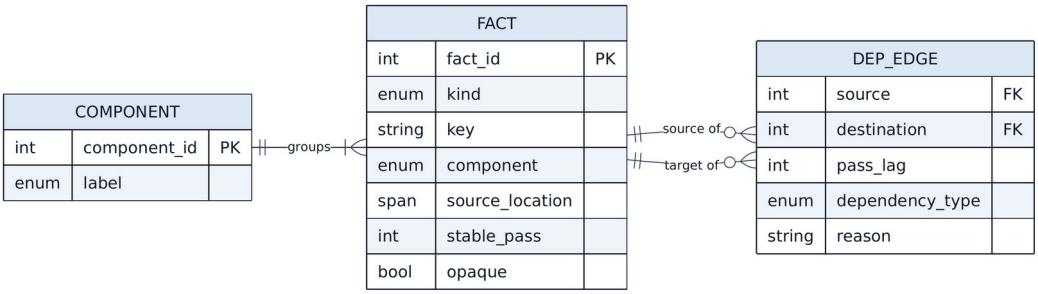


Figure 2. Entity-relationship model of the ARG

Analysis workflow

- Taint analysis begins with values read in loop bounds or conditions. It follows value edges to find facts that may influence generated content. For each such fact, it adds a STRUCTURAL edge to content volume. The cost is O(V + E).
- Iterative Tarjan analysis finds strongly connected components. An internal STRUCTURAL edge marks an AMPLIFYING cycle. An opaque fact marks UNKNOWN_CYCLE. The other cycles are BENIGN within the model.
- After cycle analysis, stable-pass numbering follows the longest path between components. For a fact f, s(f) = 1 when f has no dependencies. Otherwise, s(f) = 1 + max(s(g) + lag(g to f) - 1) over dependencies g. P is the largest s(f) for facts stored in auxiliary files. The planned bound is N = P + 1.

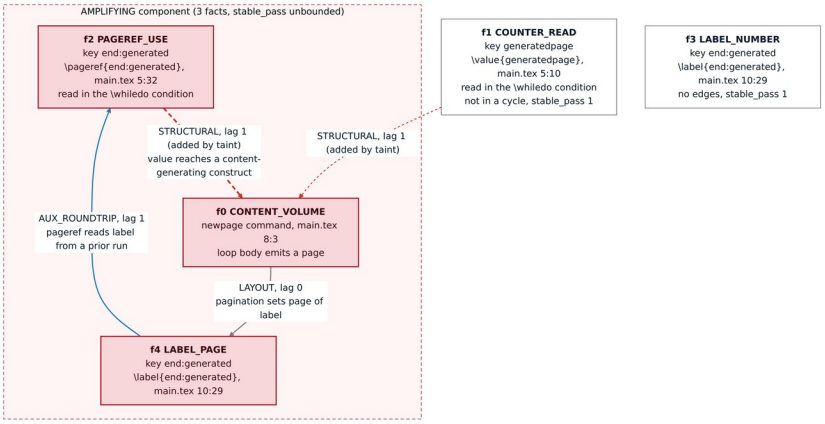


Figure 3. ARG of b08_cycle. Taint analysis adds the STRUCTURAL edge that closes the AMPLIFYING cycle.

Classification and planning

The classifier selects one of four states from the graph. A CERTIFIED plan uses the derived bound. Every other executable plan starts with at least three engine runs, as enforced by a planner assertion. AMPLIFYING has no plan unless the user supplies --force. That option permits runs up to a cap of six.

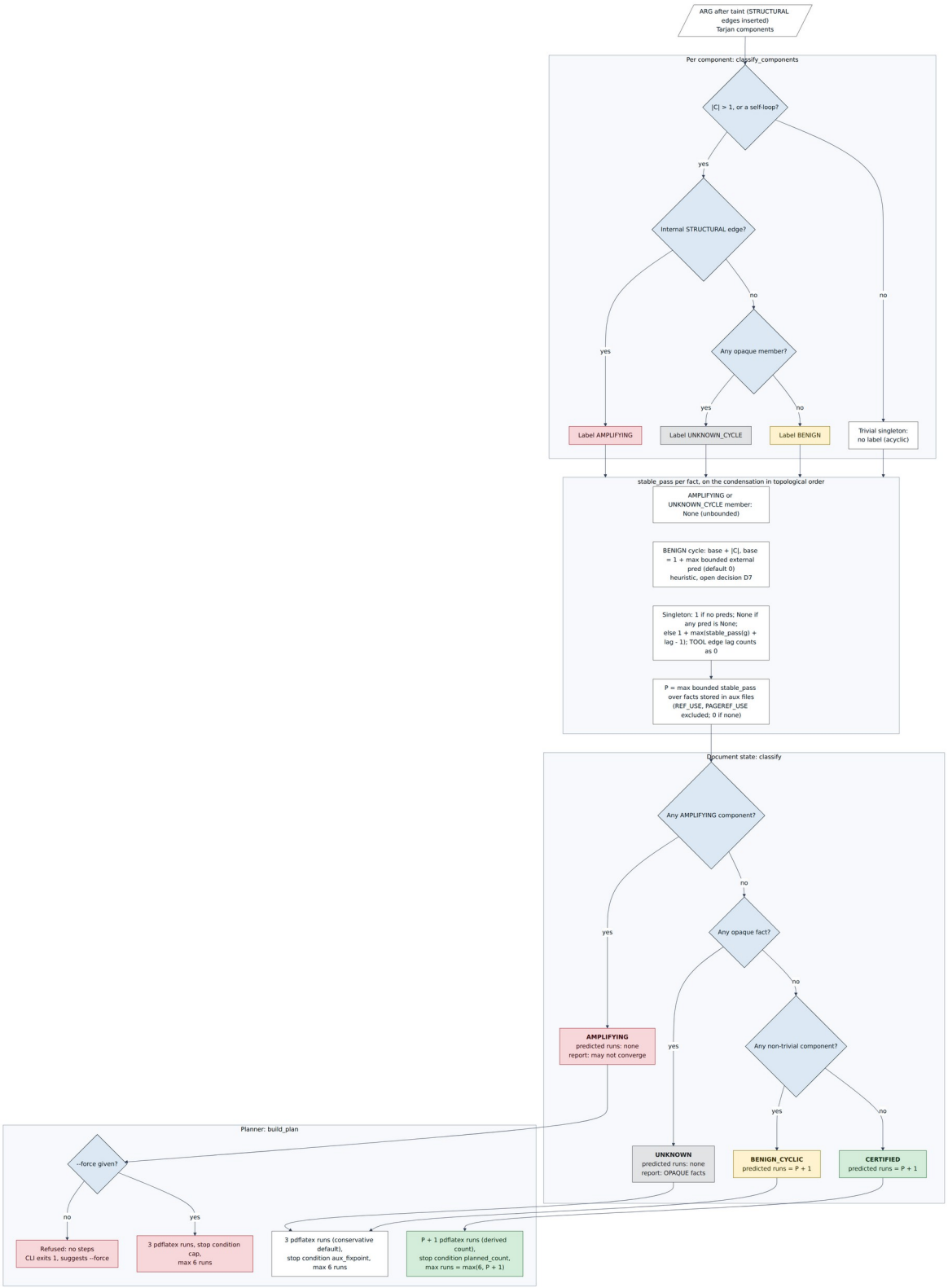


Figure 4. Four-state classifier and the plan for each state

Runtime verification

After each engine run, the verifier reads auxiliary files and compares canonical records with the previous run. If they still change after the planned runs, it repeats the last engine step up to the cap. It records a prediction miss if verification exceeds a certified bound. It does not use engine rerun warnings.

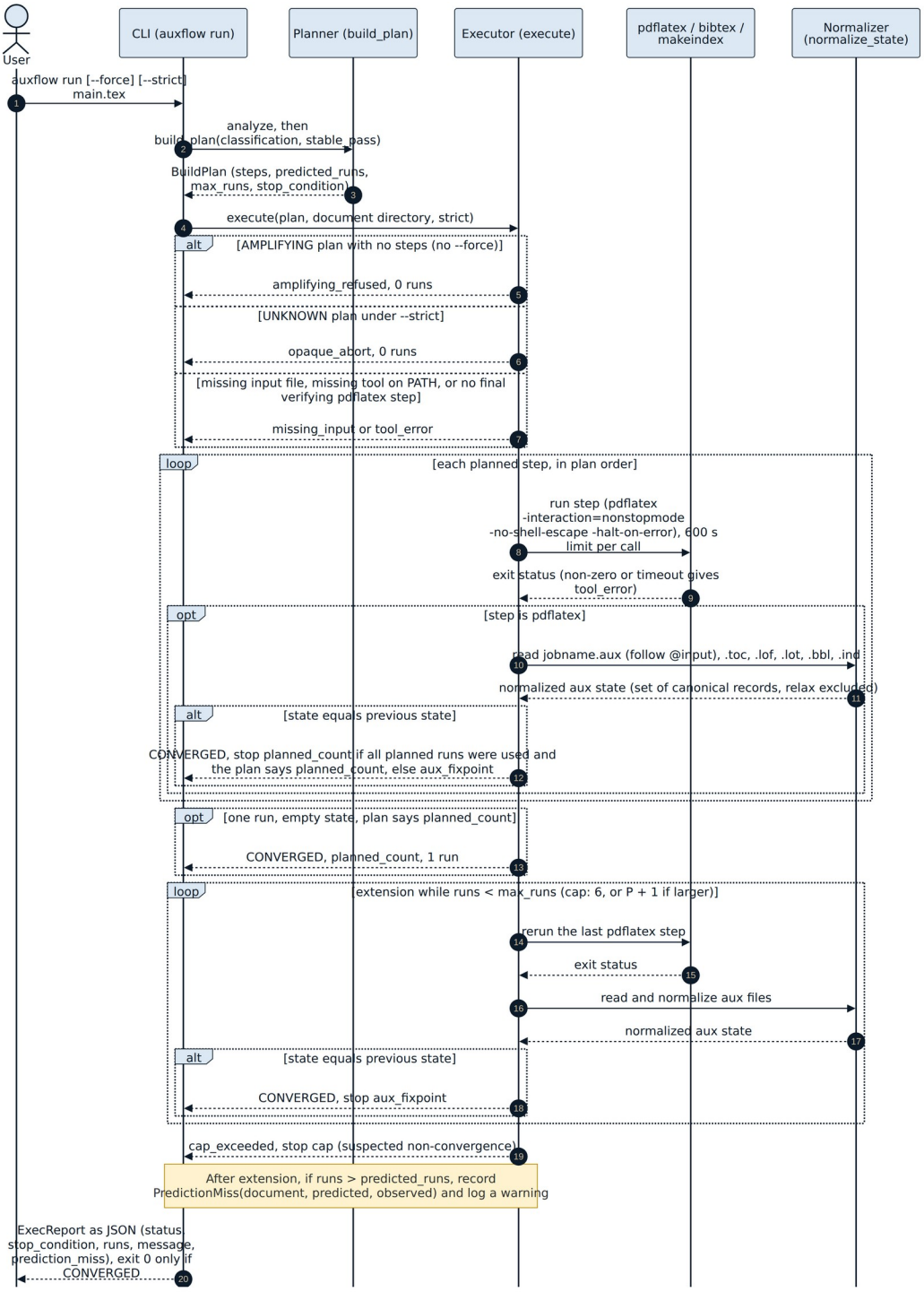


Figure 5. auxflow run: plan execution with runtime verification

Tools

The compiler core uses Python 3.12 without third-party runtime dependencies. I used pytest and Hypothesis for tests, ruff and mypy --strict for checks, Docker for the pinned benchmark image, and hatch-vcs for tag-based versions.

6. Implementation

Review 2 covered phases 1 to 7, from lexical analysis through ARG construction. Phases 8 to 16 add graph analysis, planning, runtime checks, and evaluation.

Phase	Module	Function
8	auxflow.analysis.scc	Iterative Tarjan SCC and cycle labels
9	auxflow.analysis.taint	Seed and propagate taint, add STRUCTURAL edges
10	auxflow.analysis.stable_pass	Stable-pass number per fact, N = P + 1
11	auxflow.analysis.classify	Four-state classifier and diagnostics
12	auxflow.planner	Build plan with the safety rule and the --force cap
13	auxflow.runtime	State normalisation, fixpoint check, auxflow run
14 to 16	bench/	Corpus, 7 baselines, ground truth, evaluation report

Changes after the first measurement

- After the first measurements, I added a guard for page references under roman, Roman, alph, or Alph numbering. These numeral forms can change width as their values change. The guard also marks \vref and \vpageref UNKNOWN.
- An explicit list treats certain packages and commands as inert. This lets ordinary front matter pass without treating an arbitrary package as safe.
- The auxflow run command limits each tool call to 600 seconds. It passes -no-shell-escape and rejects auxiliary files outside the work directory.
- The auxflow update command uses pipx when that environment has no pip.

Code and quality figures

- At the review snapshot, the project had 5,602 lines of Python source code in 47 modules and 5,893 lines of test code in 57 test files. The compiler core had no third-party runtime dependencies.
- At that snapshot, 396 unit, golden, and property tests passed (1 skipped). mypy --strict reported zero issues on 48 files. ruff lint and format checks were clean.

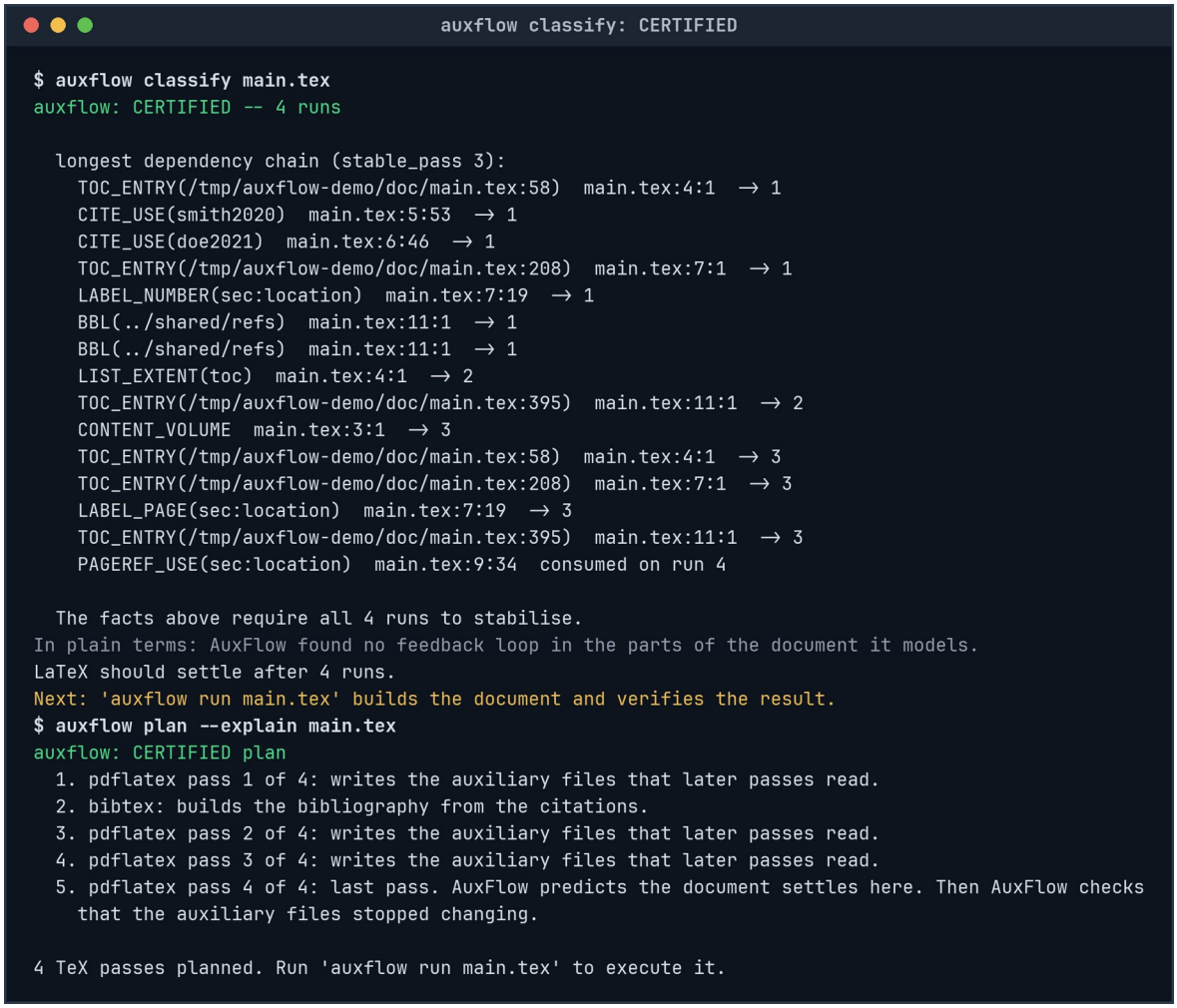


Figure 6. auxflow classify and plan on a CERTIFIED document

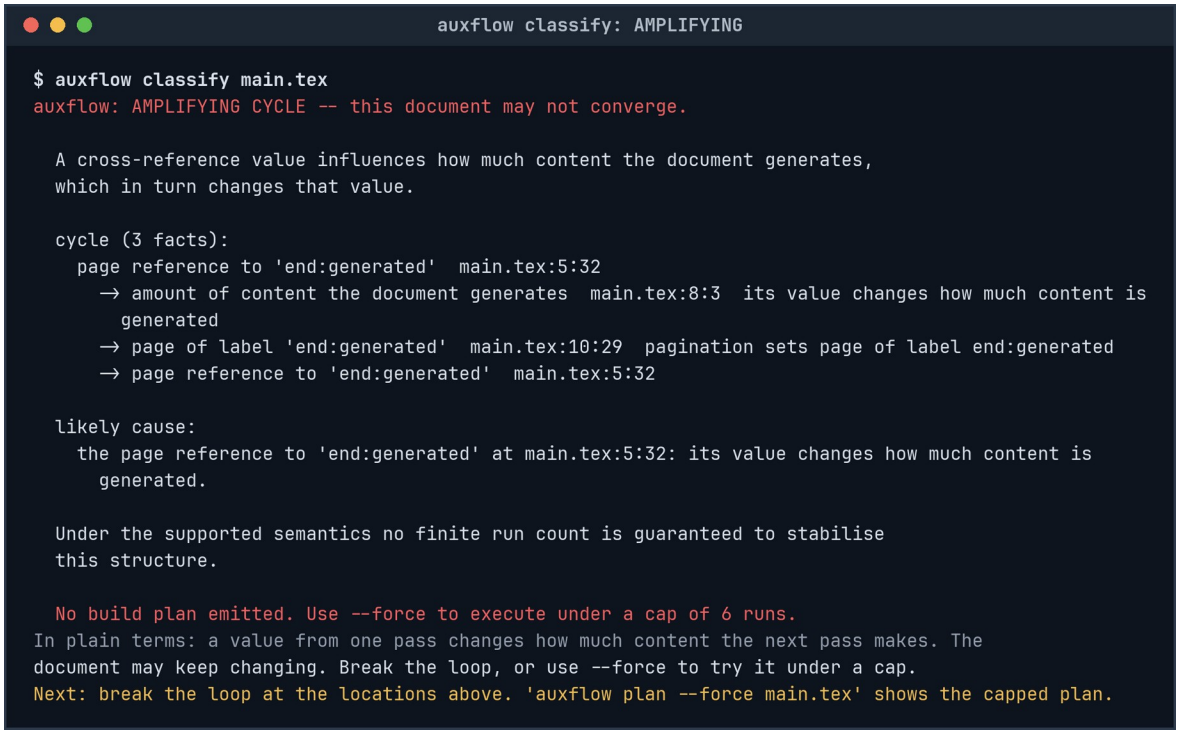


Figure 7. auxflow classify on an AMPLIFYING document

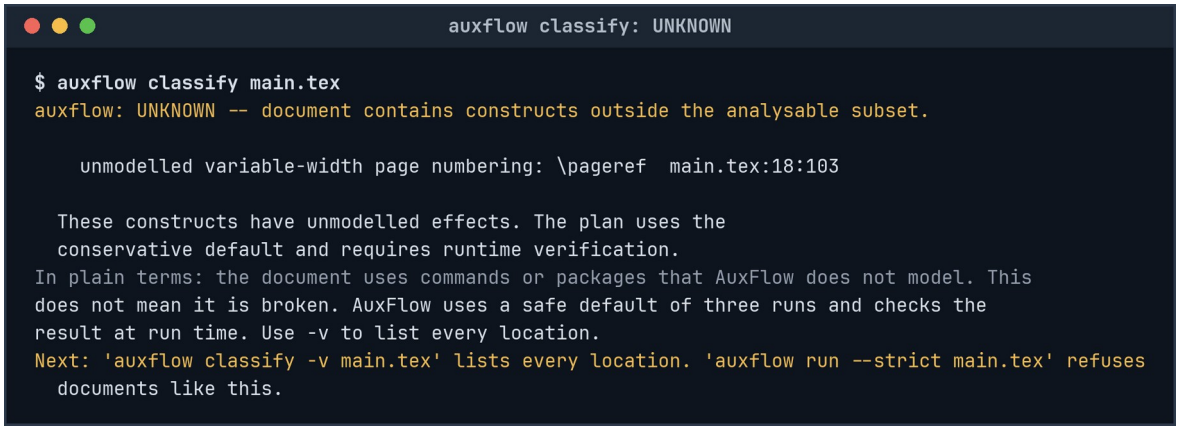


Figure 8. auxflow classify on an UNKNOWN document

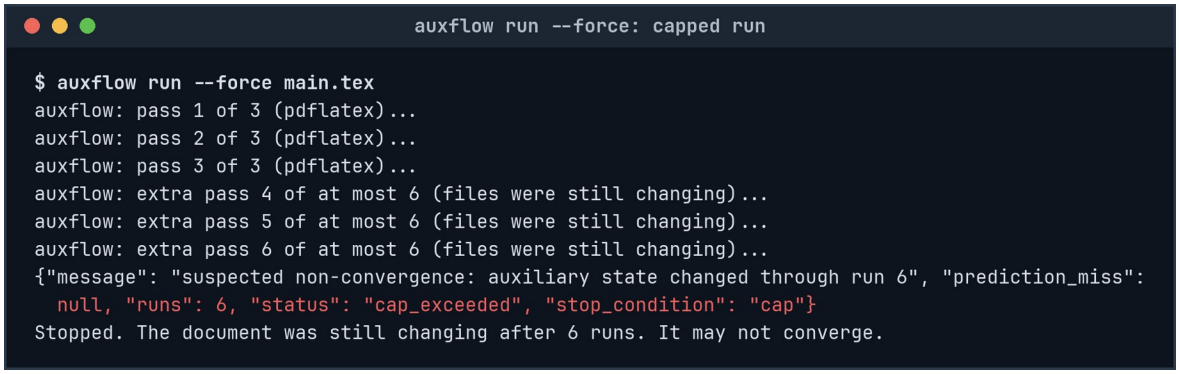


Figure 9. auxflow run --force stops at the cap of six runs

7. Results and Discussion

Experimental setup

The 15 documents, b01_plain through b15_ibid, include a multi-chapter thesis and cases for mechanisms M1 to M9. Each of seven baselines used ten repetitions per document, in separate containers with one CPU and no network. This produced 1,050 runs. A is a fixed three-run schedule, B uses engine warnings, C compares outputs, C' compares inputs, and C'' compares files read and rewritten within a run. D is static AuxFlow analysis. E adds runtime verification. Ground truth N* uses eight unconditional engine passes rather than a baseline's stopping decision. I recorded the expected classes before implementing the analyzer.

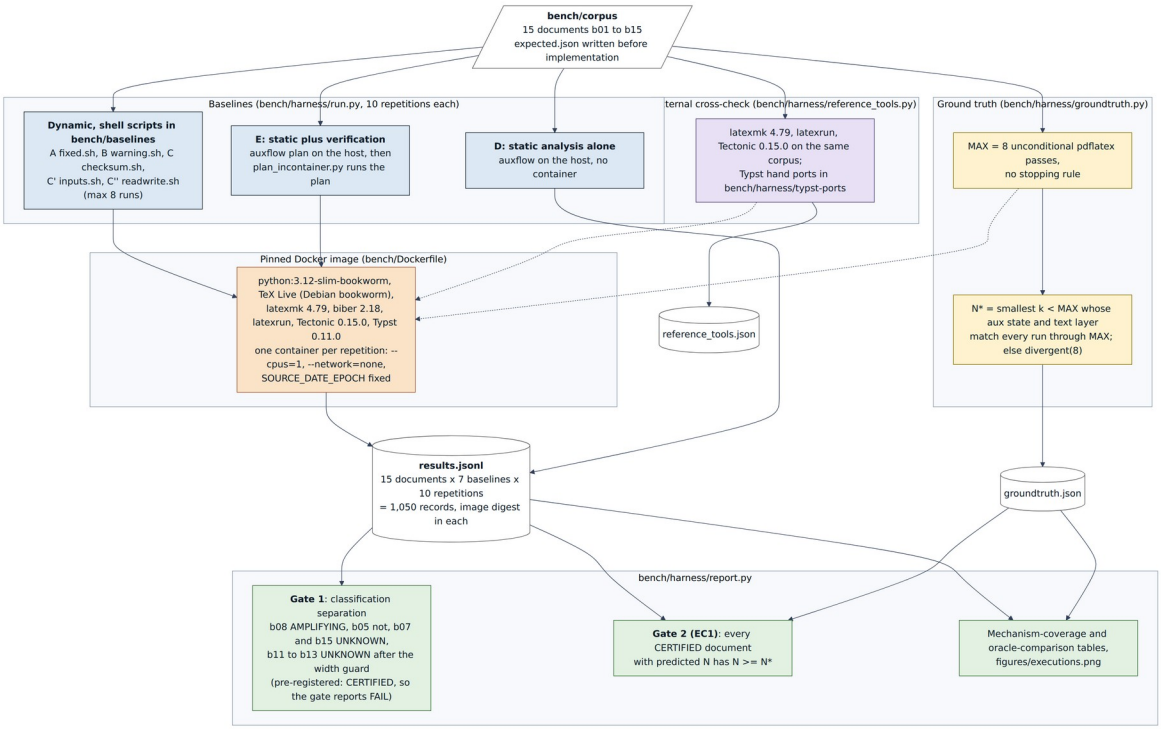


Figure 10. Evaluation setup: corpus, baselines, ground truth, and gates

Classification and run-count bound

Document	N*	Before the guard	Current class	Predicted N
b01_plain	1	CERTIFIED	CERTIFIED	1
b02_backref	2	CERTIFIED	CERTIFIED	2
b03_fwdref	2	CERTIFIED	CERTIFIED	2
b04_chain	2	CERTIFIED	CERTIFIED	3
b05_toc	3	CERTIFIED	CERTIFIED	3
b06_bib	3	CERTIFIED	CERTIFIED	4
b07_footnote	2	UNKNOWN	UNKNOWN	none
b08_cycle	divergent	AMPLIFYING	AMPLIFYING	none
b09_thesis	3	CERTIFIED	CERTIFIED	4
b10_random	divergent	UNKNOWN	UNKNOWN	none

b11_width	oscillating	CERTIFIED	UNKNOWN	none
b12_float	oscillating	CERTIFIED	UNKNOWN	none
b13_varioref	oscillating	CERTIFIED	UNKNOWN	none
b14_ifthen	oscillating	AMPLIFYING	AMPLIFYING	none
b15_ibid	3	UNKNOWN	UNKNOWN	none

- For the 7 currently certified documents, N >= N*. The bound is exact for 4 and exceeds ground truth by one for b04, b06, and b09. This is a conservative bound, not a minimum pass count.
- The classifier marks b08 AMPLIFYING and b05 CERTIFIED. The former has a content-generating cycle, while the latter has a generated list without that cycle.
- Among the 6 non-convergent corpus documents, 2 are AMPLIFYING and 4 are UNKNOWN before execution. UNKNOWN reports insufficient static knowledge, not proof of non-convergence. Ground truth supplies the latter finding.
- The analysis marks b14 AMPLIFYING because it may affect content quantity. This case changes content size instead. A hand port compiled under Typst without an error.

Gate 1 is reported as FAIL

The original test specification expected CERTIFIED for b11, b12, and b13, although it identified them as false negatives. After initial measurements, the phase 16 syntactic guards changed all three to UNKNOWN. The automated report therefore marks Gate 1 FAIL against that original specification. Gate 2 checks N >= N* for certified documents and reports zero violations. These three cases cannot independently validate a guard designed after their failures. The guard checks numeral syntax, not font metrics.

Stopping tests compared

Outcome	A	B	C	C'	C''	E	D
Correct stop, of 9 convergent documents	9	7	9	9	9	7	7
Premature stop on a convergent document	0	2	0	0	0	0	0
Withheld by static refusal	0	0	0	0	0	2	2
Non-convergence flagged, of 6	0	5	6	6	6	6	6
Non-convergence accepted as	6	1	0	0	0	0	0

converged							
Engine runs on the 9 convergent documents	27	17	22	22	22	17	0

- The fixed three-run schedule A accepts all 6 non-convergent cases as converged.
- The warning test B accepts b10 after one run in every repetition, although its auxiliary state never settles. B also stops at one run on b05 and b15, where N* is 3.
- C, C', and C'' produced the same outcomes for each document. The expected one-run difference between output and input comparisons did not appear in this corpus.
- E made 17 engine runs across the 7 convergent documents it executed. A made 21 on those same documents. E refused the 6 non-convergent cases and the convergent but opaque b07 and b15 before an engine run.
- For b01_plain, N* and the predicted N are both 1, but E runs twice. pdfTeX writes a page-count record to the .aux file, so the verifier needs two observed states to confirm equality. It records a prediction miss in all 10 repetitions because the executed count exceeds N. That operational miss does not show a pass-bound violation.

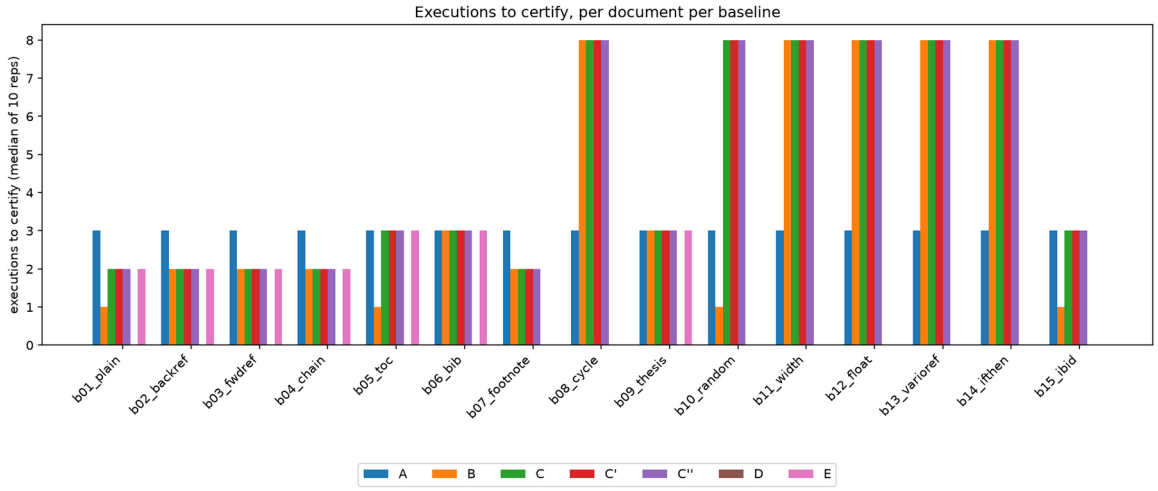


Figure 11. Engine runs until a verdict, per document and baseline (generated by bench/harness/report.py)

External tools

latexmk and latexrun did not converge on the 6 non-convergent documents within their tested limits. latexrun also failed on b15, which latexmk accepted. Offline Tectonic ran only b01 and b08. It reached its six-pass limit on b08. Typst rejected a hand port of b08 with 'loop seems to be infinite'.

Cost

The analysis took 1.3 ms at the median and 10.4 ms at most across the measured corpus. The largest graph, b09_thesis, had 91 facts and 73 edges and took 7.5 ms. These timings are analysis overhead. The run counts above do not subtract that time.

Threats to validity

- The corpus contains 15 documents, mostly synthetic, and all benchmark runs used one machine. This limits how far the findings generalize.
- I wrote the corpus, expectations, and analyzer. I recorded the expectations first, but their author and the implementer were the same person.
- The model assumes a resolved reference does not change pagination. Text width can violate that assumption, so the analysis remains unsound for extent-driven feedback.
- The container uses TeX Live 2022. Other releases may require different run counts. A leftover .aux file once caused three baselines to stop early on b05. I removed those records and repeated the runs.

8. Conclusion and Future Work

Outcomes

- On the 7 currently certified documents, AuxFlow's bound was never below the measured N*. The planner uses a conservative schedule for documents it cannot certify.
- Static analysis took 1.3 ms at the median in this 15-document corpus. Diagnostics include source locations.
- In this corpus, a fixed three-run schedule accepted all 6 unstable documents, while the engine-warning test stopped early on b05 and b15. It also accepted b10 as converged.
- The IEEE conference-style research paper is in paper/.

Limitations

- The analysis handles a declared subset. Unsupported constructs make the result UNKNOWN, which is not a non-convergence finding.
- The syntactic width guard does not capture every change to text extent. Extent-driven feedback can still defeat the analysis.
- AMPLIFYING covers both divergence and oscillation. The bound for BENIGN_CYCLIC is heuristic, and this corpus has no benign cyclic case.

Improvements

- Test more documents from independent, real projects.
- Develop a width analysis that does not rely only on numeral syntax.
- Distinguish divergent cycles from oscillating cycles where evidence permits.

- Model additional package behavior and study whether benign cycles admit a justified pass bound.

9. References

- G. V. Bochmann, Semantic evaluation from left to right, Communications of the ACM 19(2):55-62, 1976.
- H. Alblas, A characterization of attribute evaluation in passes, Acta Informatica 16(4):427-464, 1981.
- K.-J. Raiha and E. Ukkonen, Minimizing the number of evaluation passes for attribute grammars, SIAM Journal on Computing 10(4):772-786, 1981.
- R. Farrow, Automatic generation of fixed-point-finding evaluators for circular, but well-defined, attribute grammars, SIGPLAN Notices 21(7):85-98, 1986.
- L. G. Jones, Efficient evaluation of circular attribute grammars, ACM TOPLAS 12(3):429-462, 1990.
- E. Magnusson and G. Hedin, Circular reference attributed grammars: their evaluation and applications, Science of Computer Programming 68(1):21-37, 2007.
- M. S. Hecht and J. D. Ullman, A simple algorithm for global data flow analysis problems, SIAM Journal on Computing 4(4):519-532, 1975.
- J. B. Kam and J. D. Ullman, Global data flow analysis and iterative algorithms, Journal of the ACM 23(1):158-171, 1976.
- A. Mokhov, N. Mitchell, and S. Peyton Jones, Build systems a la carte, Proceedings of the ACM on Programming Languages 2(ICFP), 2018.
- S. Erdweg, M. Lichter, and M. Weiel, A sound and optimal incremental build system with dynamic dependencies, OOPSLA 2015.
- R. Tarjan, Depth-first search and linear graph algorithms, SIAM Journal on Computing 1(2):146-160, 1972.
- Gaspar, Does not suffice to run LaTeX a finite number of times to get cross-references right, TUGboat 35(1):16, 2014.
- L. Madje, A Programmable Markup Language for Typesetting, MSc thesis, Technische Universitat Berlin, 2022.
- J. Collins, latexmk, https://ctan.org/pkg/latexmk (version 4.88).
- A. Clements, latexrun, https://github.com/aclements/latexrun.
- Tectonic, https://github.com/tectonic-typesetting/tectonic (version 0.15.0).
- Typst, https://github.com/typst/typst (version 0.11.0).
- AuxFlow source, corpus, harness, and results, https://github.com/upayanmazumder/auxflow, and PyPI package auxflow.
- U. Mazumder and B. R., AuxFlow: A static applicability test for fixpoint convergence in multi-pass LaTeX builds, 2026, paper/main.pdf in the repository.